V2 Book: Start Here

One book, one index, one method. This is the single source of truth for auditing, fixing, and productizing the PRM/CRM as a configurable, self-service, intercompany ecosystem. It replaces V1 (kept in the Archive tab) and is written for people to read and for AI to load in pieces.

How to use this book

- **People:** read this tab, then open the Part you need. Every chapter opens with a purpose, a load-when line, and what it produces.
- **AI sessions:** load this tab plus only the chapters the task needs (Context packs below). Never rely on memory of earlier sessions; re-read the chapter and the ledger first.
- **Changing the book:** edit the chapter, add a change-log line, update the ledger. Superseded text moves to the Archive tab; nothing is deleted.
- **Status rule:** every gap is *suspected* until evidence from code, data, or a running system confirms it.

AI operating contract (paste at the start of every session)

1. Role: principal PRM/CRM architect, channel-operations expert, and independent auditor; expert in C#, ASP.NET Core on .NET 10, DDD, EF Core, and React with TypeScript.
2. Domain model first, then API, then UI.
3. One slice at a time (C19). State scope, chapters loaded, ledger IDs, and acceptance tests before changing anything.
4. Evidence over assumption: cite the file, class, table, route, config key, or observed behavior, and label the evidence class (Conventions below).
5. Configuration over branching: no behavior switched on channel, tenant, or partner type in code without an ADR.
6. Generate, do not copy: roles, nav, doc matrices, and demo data derive from the Access Registry (C14) and the Journey Spine (C16).
7. Never weaken tenant isolation, authorization, consent, or audit to ship a feature.
8. End every slice by updating the ledger, affected chapters, change log, lessons learned, and model log.

9. New ideas go to the parking lot (C21), not into the current slice.
10. For security, data-model, and release-gate work, the reviewer is a different session, preferably a different model, than the author.

Book map (index)

ID	Tab	Chapter	Load when	Produces
C01	Part I	Mission and principles	Starting any session	Shared frame
C02	Part I	Five structures	Modeling relationships, rules, ownership	Structure decisions
C03	Part I	Organizations, relationships, hats	Multi-role companies, customers who are also vendors	Org/hat model
C04	Part I	Franchise and master license	Sub-companies, license pools	Ownership-tree model
C05	Part I	Channel templates and vertical overlays	Defining or auditing a channel or industry	Rule cards
C06	Part I	Deal lifecycle, conflict, attribution	Lead, registration, opportunity, credit	State model
C07	Part II	Target architecture	Layering, modules, code review	Architecture rules
C08	Part II	Configuration inheritance	Rules, overlays, exceptions	Resolver spec
C09	Part II	Object model rationalization	The 265+ tables, drift	Entity catalog
C10	Part II	Work inbox and propagation	Tasks, reminders, notifications	Propagation map
C11	Part II	Integration: Stateless and Activepieces	Workflows, connectors	Integration catalog
C12	Part II	Security, governance, ADRs	Access, audit, privacy, guardrails	ADR set

ID	Tab	Chapter	Load when	Produces
C13	Part II	Content, RFx, and AI	Publishing, RFQ/RFP, AI features	AI controls
C14	Part III	Access Registry, roles, concierge	Roles, hats, permissions	Registry
C15	Part III	Navigation and hats	Menu, role switcher	Module registry
C16	Part III	Journey Spine	Journeys, workflows, tests	Spine records
C17	Part III	Visual language	Colors, icons, role chip	Tokens
C18	Part III	Demo kit	Demos	Scripts and swimlanes
C19	Part IV	Delivery cycle and phases	Planning any work	Slice plan
C20	Part IV	Ledger and finding register	Logging findings	Ledger rows
C21	Part IV	Anti-drift, lessons, doc governance	Keeping docs and code aligned	Updates
C22	Part IV	Release gates and KPIs	Go-live decisions	Gate results
C23	Part V	Tools and model routing	Choosing a tool or model	Routing
C24	Part V	Toolchain, skills, prompts	Setup and reuse	Installs, prompts
C25	Part V	Evidence and sources	Verifying claims	Source list

Context packs (what to load)

Task	Load	Suggested model
Inventory repo, docs, demo (Phase 0)	C09, C14, C19, C20	Sonnet orchestrates, Haiku fans out
Design org, hat, franchise model	C02 to C04, C08	Opus
Fix deal registration flow	C06, C10, C16, C20	Opus designs, Sonnet implements
Build nav, roles, demo switcher	C14 to C18	Sonnet
Integration and Activepieces	C11, C12	Opus
RFx and AI features	C12, C13	Sonnet; Opus for hard drafting
Release review	C12, C20, C22	Fable 5.1 or Opus, fresh session

Conventions

Item	Rule
IDs	`AUD-<area>-nnn` finding, `FEA-<area>-nnn` feature, `ADR-nnn` decision, `JRN-nnn` journey, `SLC-nnn` slice, `LL-nnn` lesson
Areas	ARCH, DOM, CFG, CHN, COL, NAV, JRN, SEC, DATA, INT, AI, UX, OPS, DOC
Evidence classes	Verified in code, Verified in running system, Vendor documentation, Vendor marketing, Third-party listing, Industry practice, Recommendation
Finding status	Suspected, Confirmed, In progress, Fixed, Verified, Won't fix, Superseded
Remedy type	Configuration, Incremental development, Integration work, Foundational redesign

What changed from V1

Change	Why
24 numbered sections plus two addenda became 25 chapters in five Parts with load-when triggers	Humans scan; AI loads only what it needs
Four structures became five (added delegation)	The concierge role needs its own model
Roles, nav, and personas were spread over four sections; now C14 to C15	One source of truth
Vertical overlays and compliance gates added	Industry research shows regulated and dealer-heavy verticals differ
Model and tool routing added (C23)	Cost, speed, and review quality
Finding register seeded with all suspected gaps (C20)	Work starts from one list
Evidence classes enforced everywhere	Marketing claims are not findings

Open questions for the owner

- Which design system is the target: Material 3, Salesforce Lightning, or Adobe Spectrum?
- Which verticals are the first customers in? (C05 overlays are prioritized by this.)
- Which CRM or ERP systems must be integrated first?
- Is the repository and a running demo available to the AI sessions, and under what data-handling rules?